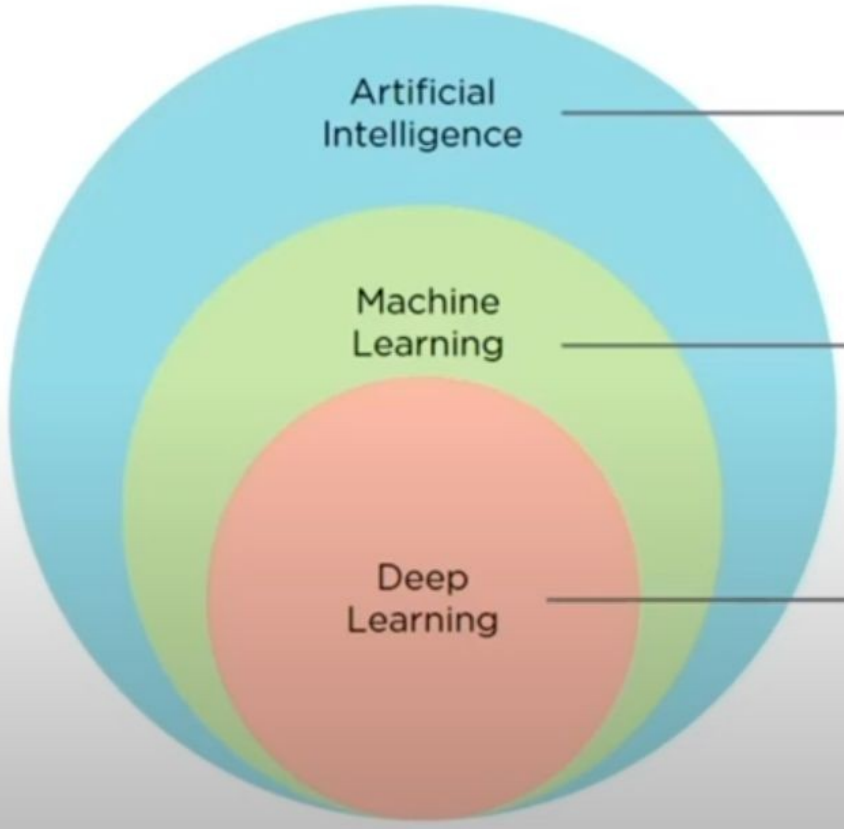
Deep Learning

By

Dr. Atif Jan

What is Deep Learning ?

Deep Learning is a subfield of Machine Learning that deals with algorithms inspired by the structure and function of the brain



Ability of a machine to imitate intelligent human behavior

Application of AI that allows a system to automatically learn and improve from experience

Application of Machine Learning that uses complex algorithms and deep neural nets to train a model

Why Do We Need Deep Learning ?

Process huge amount of data

Machine Learning algorithms work with huge amount of structured data but Deep Learning algorithms can work with enormous amount of structured and unstructured data

Perform complex algorithms

Machine Learning algorithms cannot perform complex operations, to do that we need Deep Learning algorithms

To achieve the best performance with large amount of data

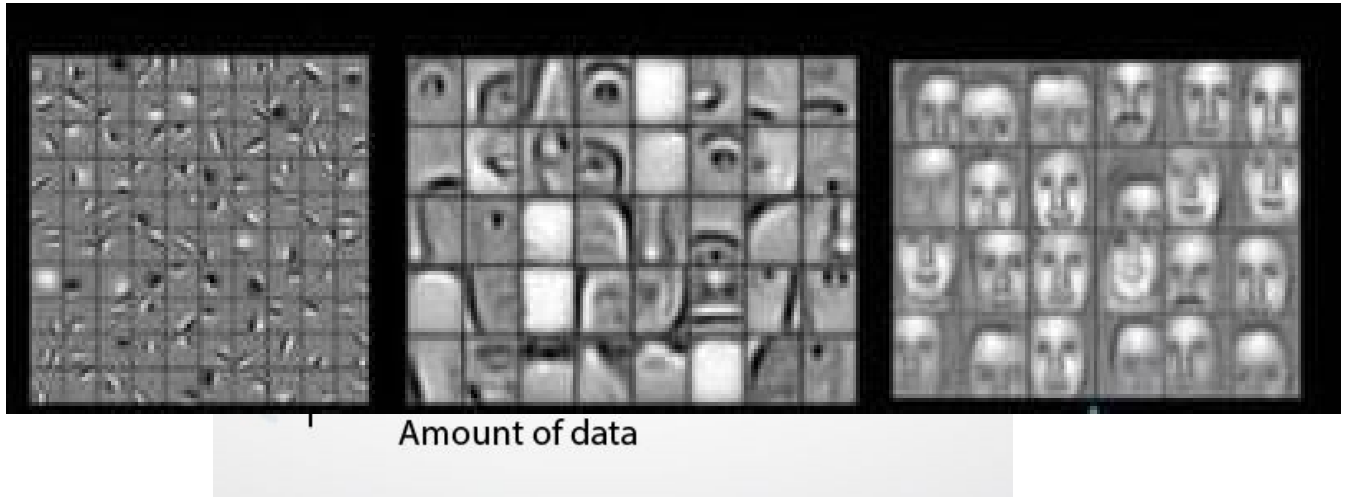
As the amount of data increases, the performance of Machine Learning algorithms decreases, to make sure the performance of a model is good, we need Deep Learning

Feature Extraction

Machine Learning algorithms extract patterns based on labelled sample data, while Deep Learning algorithms take large volumes of data as input, analyze the input to extract features out of an object and identifies similar objects

Deep Learning Vs Machine Learning

- Data Dependency
- Hardware Dependency
- Training Time
- Feature Selection



Why Now ?

- 1960’s Alan Turing
- 2012 got famous :
  - Datasets
  - Hardware
  - Frameworks
  - Architecture

Applications of Deep Learning

Cancer Detection

Deep Learning helps to detect cancerous tumors in the human body

Robot Navigation

Deep Learning is used to train robots to perform human tasks

Autonomous Driving Cars

Distinguish between different types of objects, people, road signs and drive without human intervention

Applications of Deep Learning

Machine Translation

Given a word phrase or sentence in one language, automatically translates it into another language

Music Composition

Deep Neural Nets can be used to produce music by making computers learn the pattern in a composition

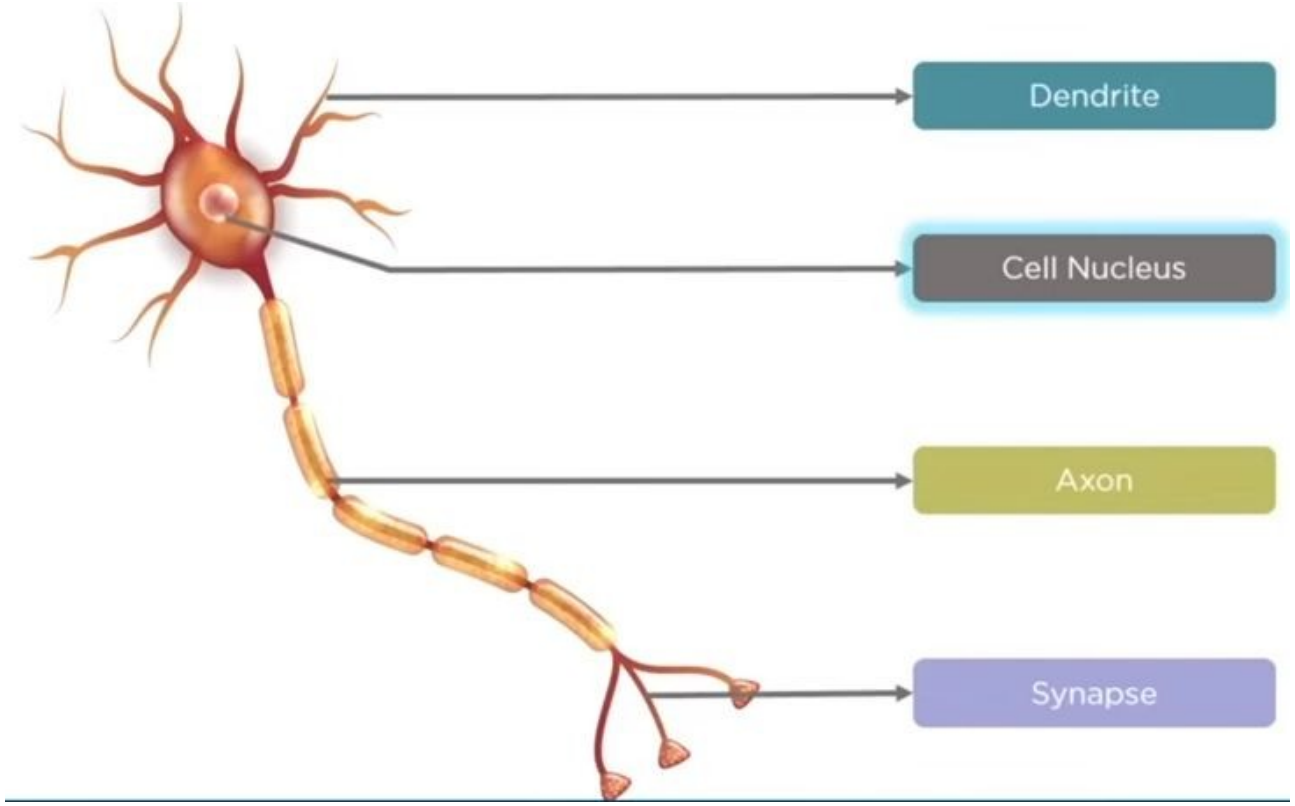
Applications of Deep Learning

Colorization of images

Use the object and their context in the photograph to color the image

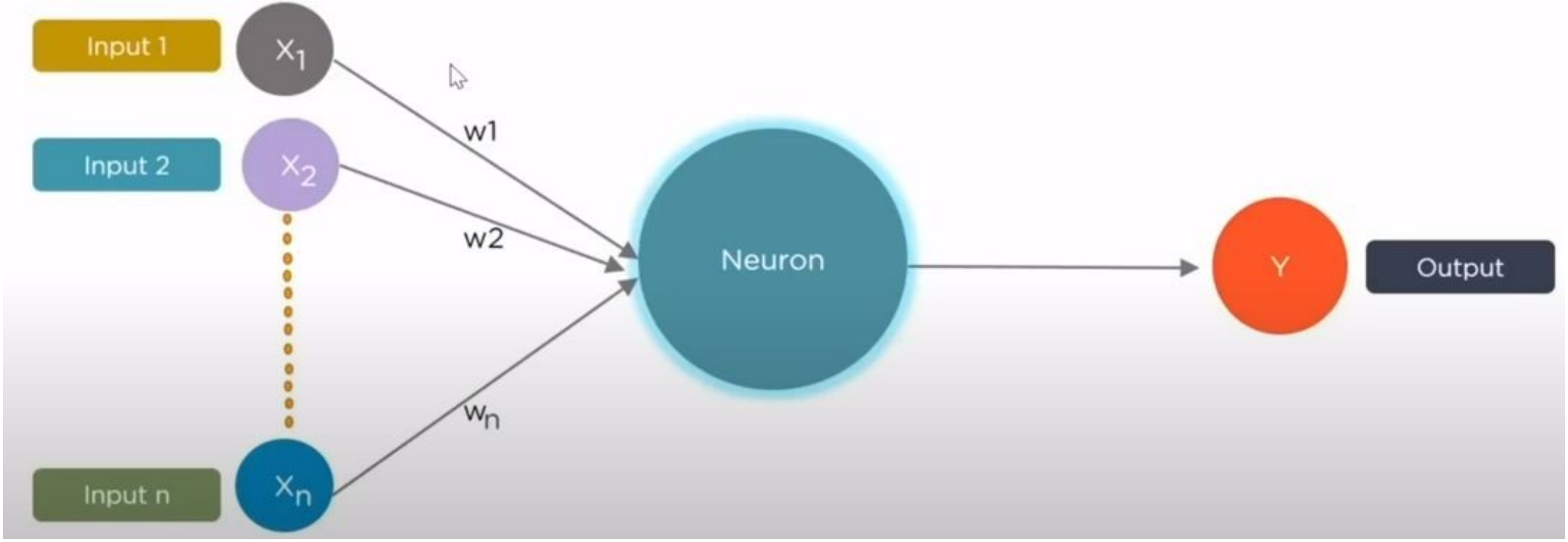
What is Neural Network ?

Deep Learning is based on the functioning of a human brain, lets understand how does a Biological Neural Network look like

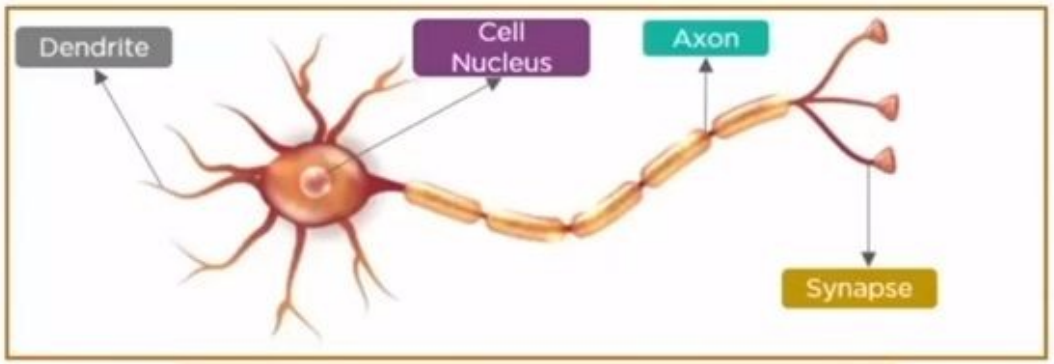


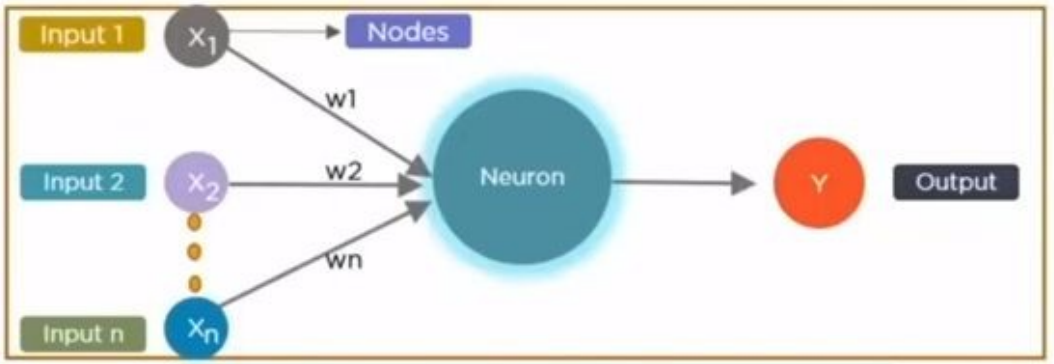
What is Neural Network ?

Deep Learning is based on the functioning of a human brain, lets understand how does an Artificial Neural Network look like



Biological Neuron Vs Artificial Neuron

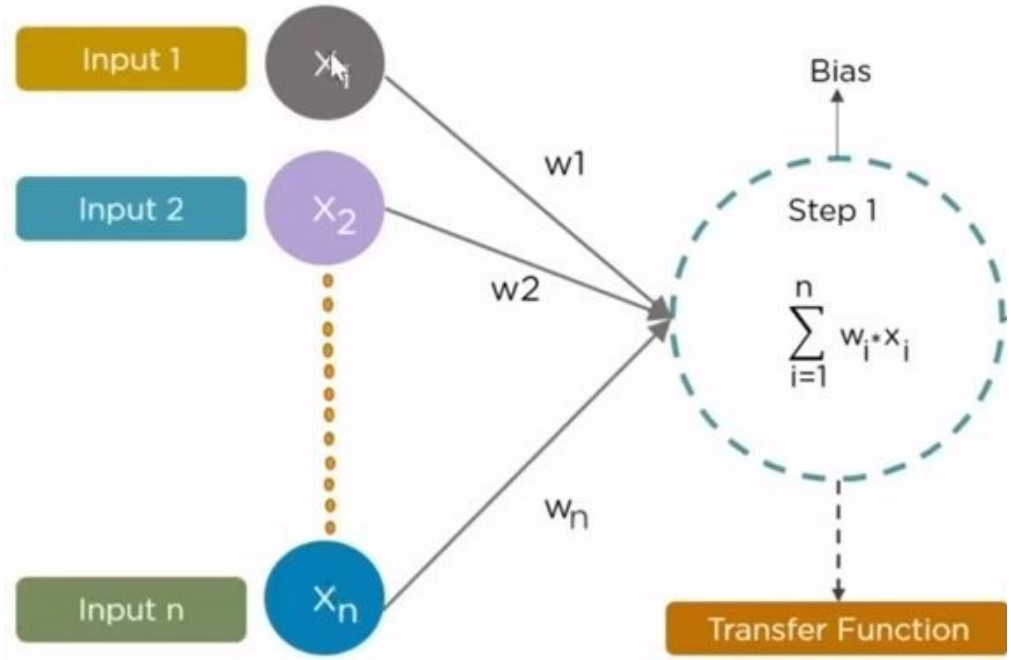




Biological Neuron	Artificial Neuron
Dendrites	Inputs
Cell Nucleus	Nodes
Synapse	Weights
Axon	Output

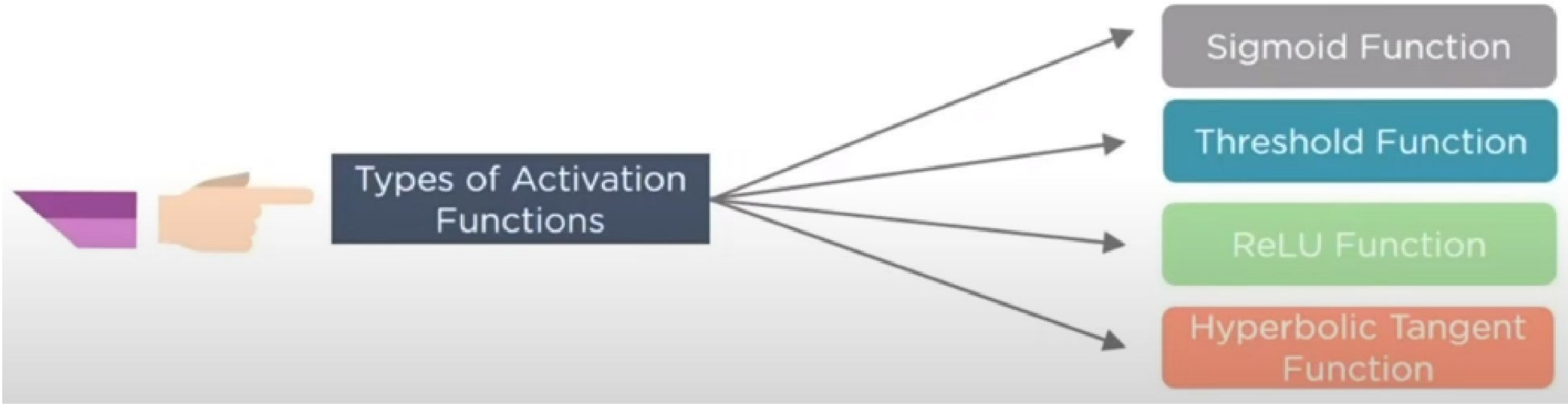
What is a Neural Network ?

Second step in the process is to pass the calculated sum as an input to the activation function to generate the output



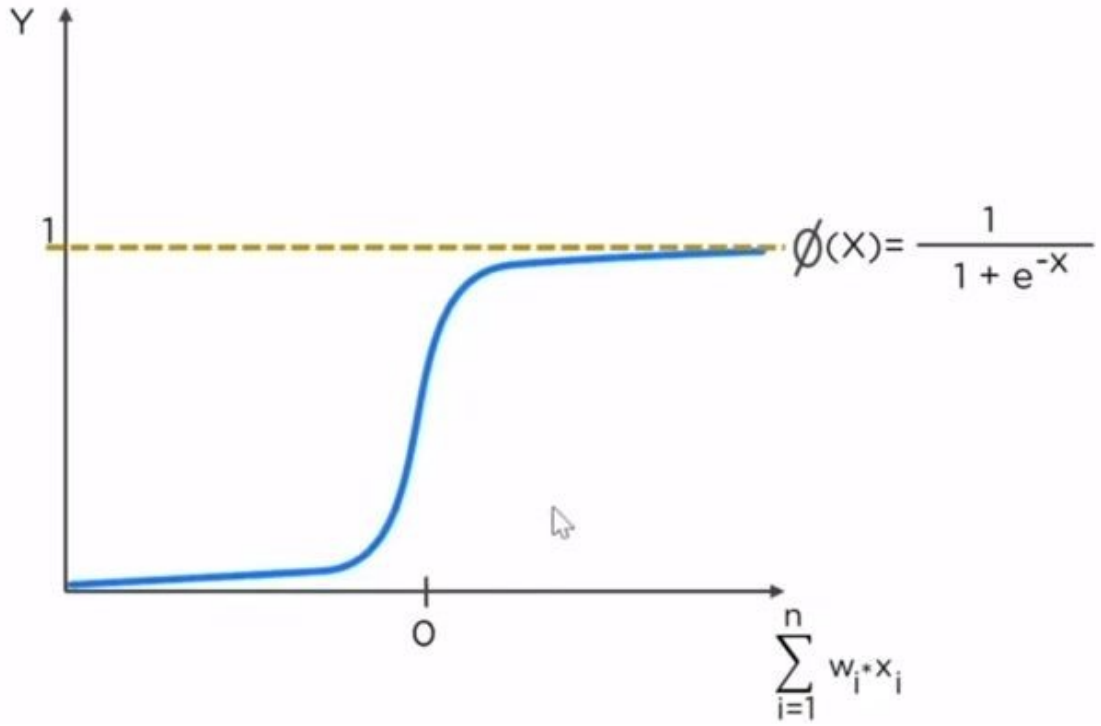
Activation functions

An activation function takes the "weighted sum of input plus the bias" as the input to the function and decides whether it should be fired or not



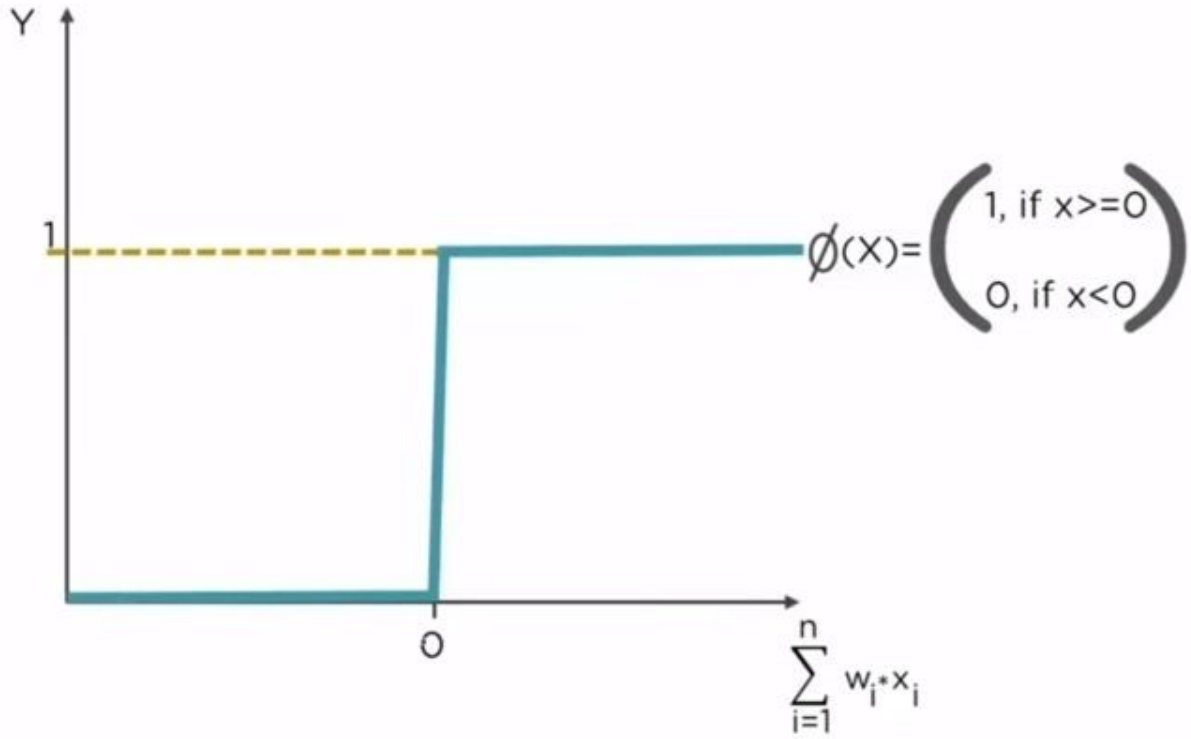
Activation functions

Sigmoid Function → Used for models where we have to predict the probability as an output. It exists between 0 and 1.



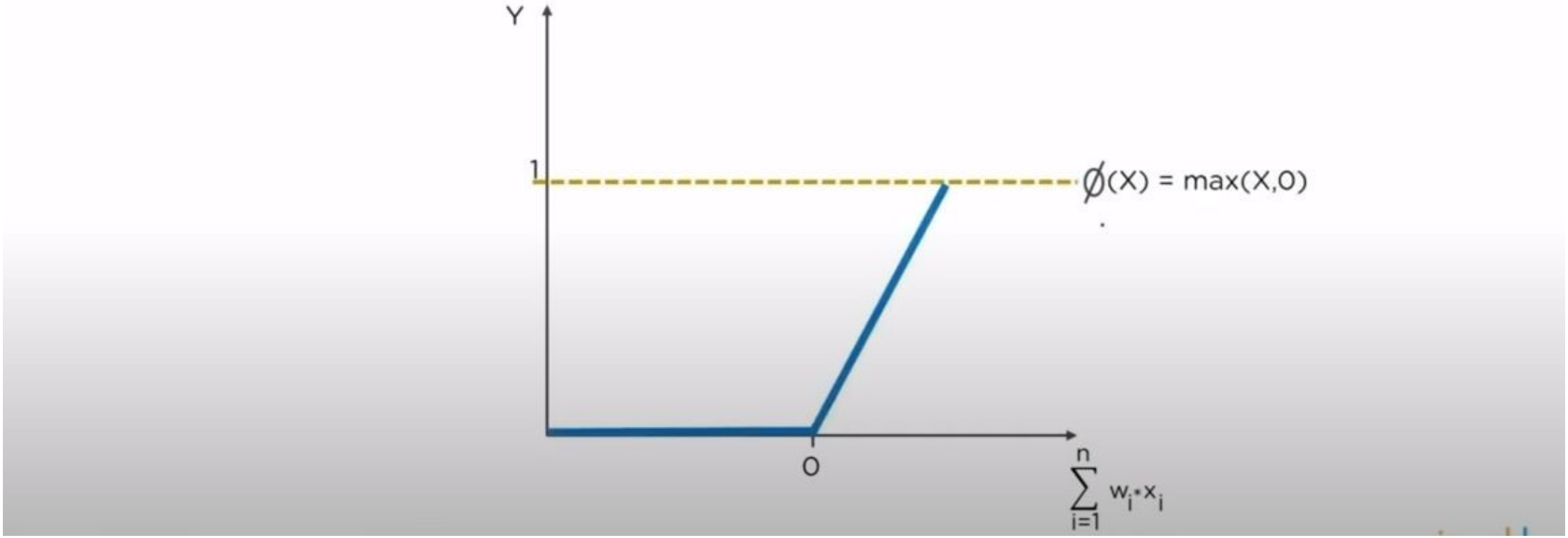
Activation functions

Threshold Function → It is a threshold based activation function. If Y value is greater than a certain value, the function is activated and fired else not.



Activation functions

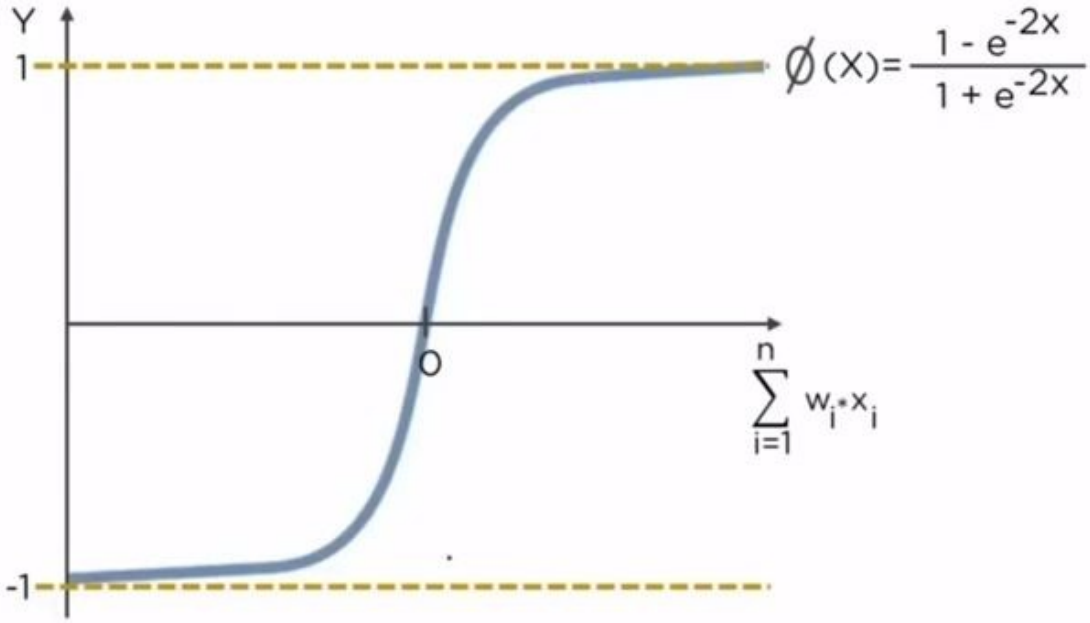
ReLU Function → It is the most widely used Activation function and gives an output of X if X is positive and 0 otherwise



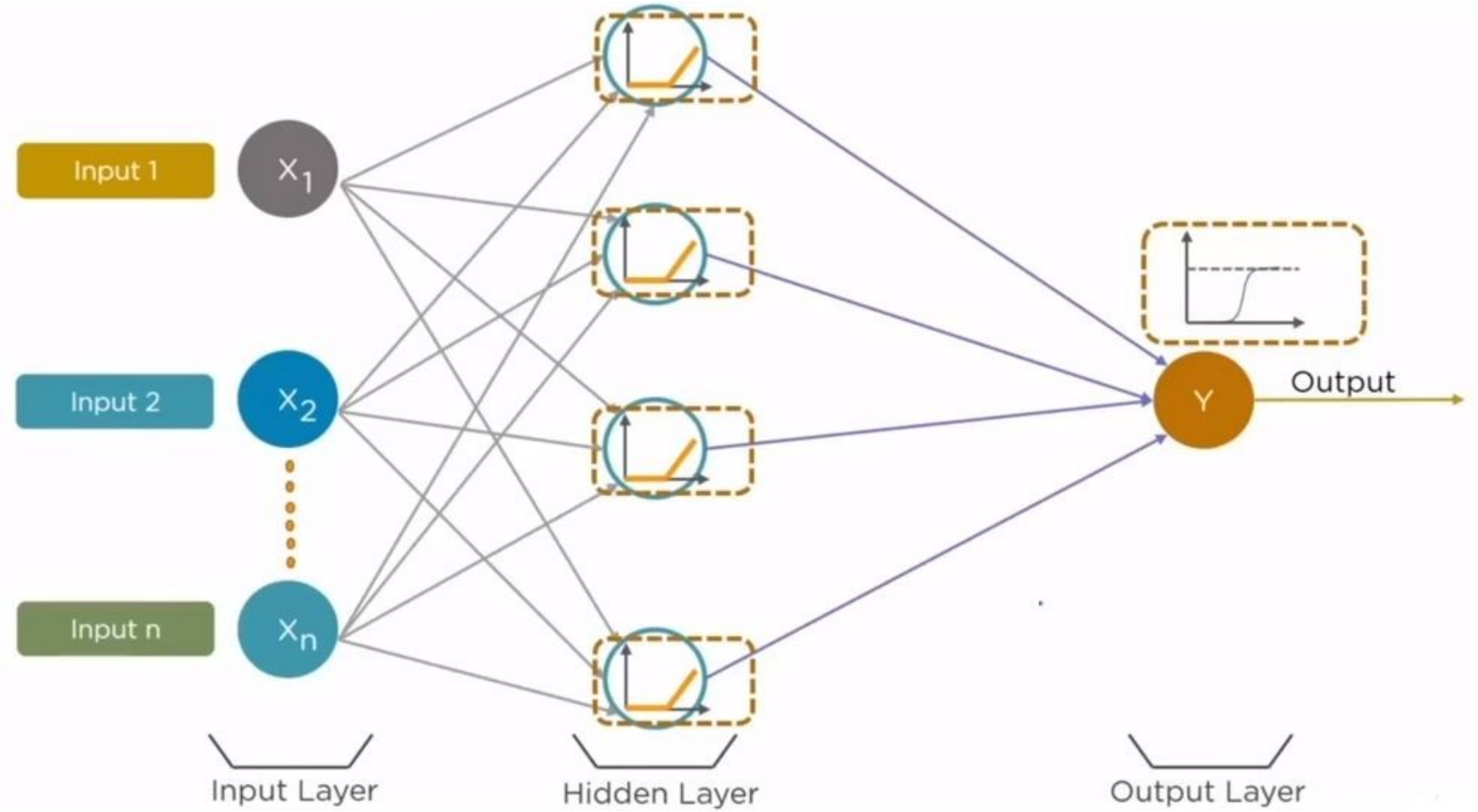
Activation functions

Hyperbolic Tangent Function →

This function is similar to Sigmoid function and is bound to range (-1, 1)

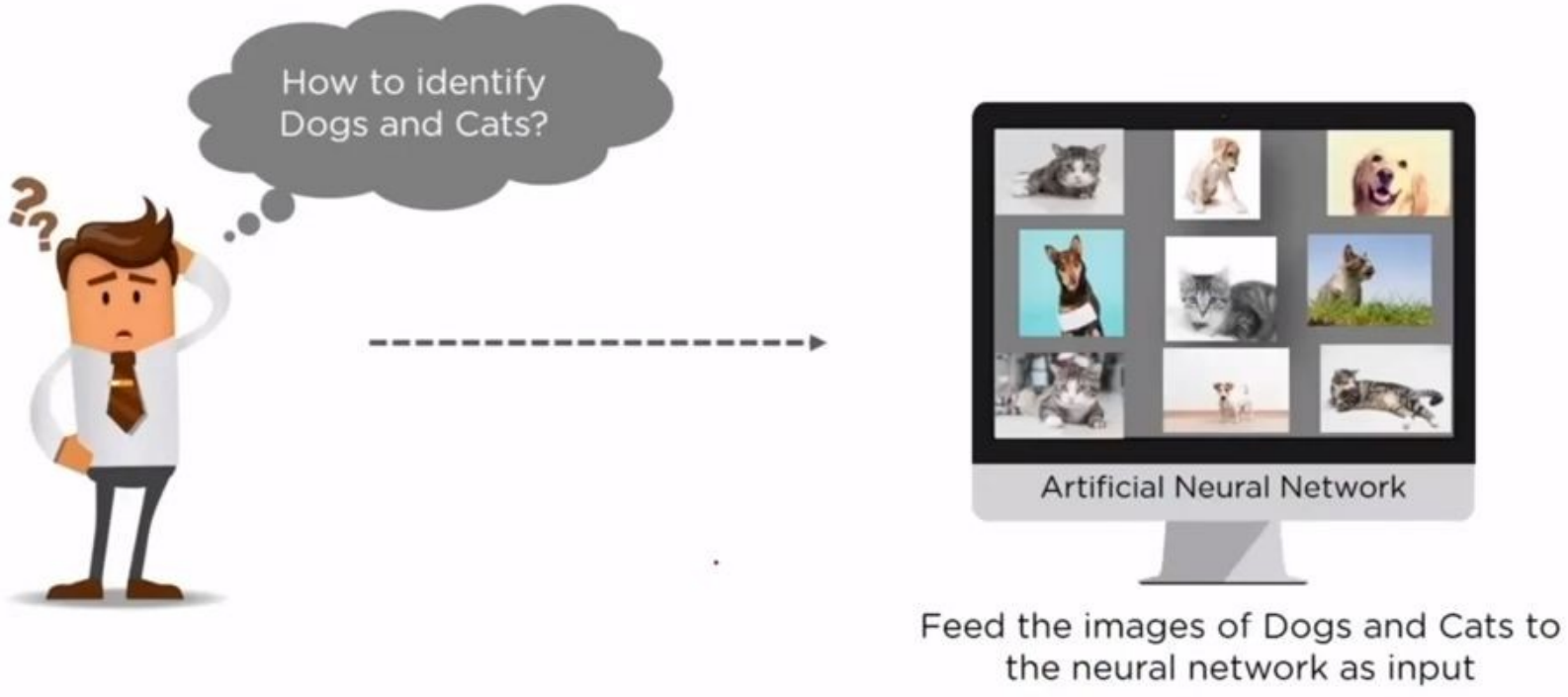


Activation functions

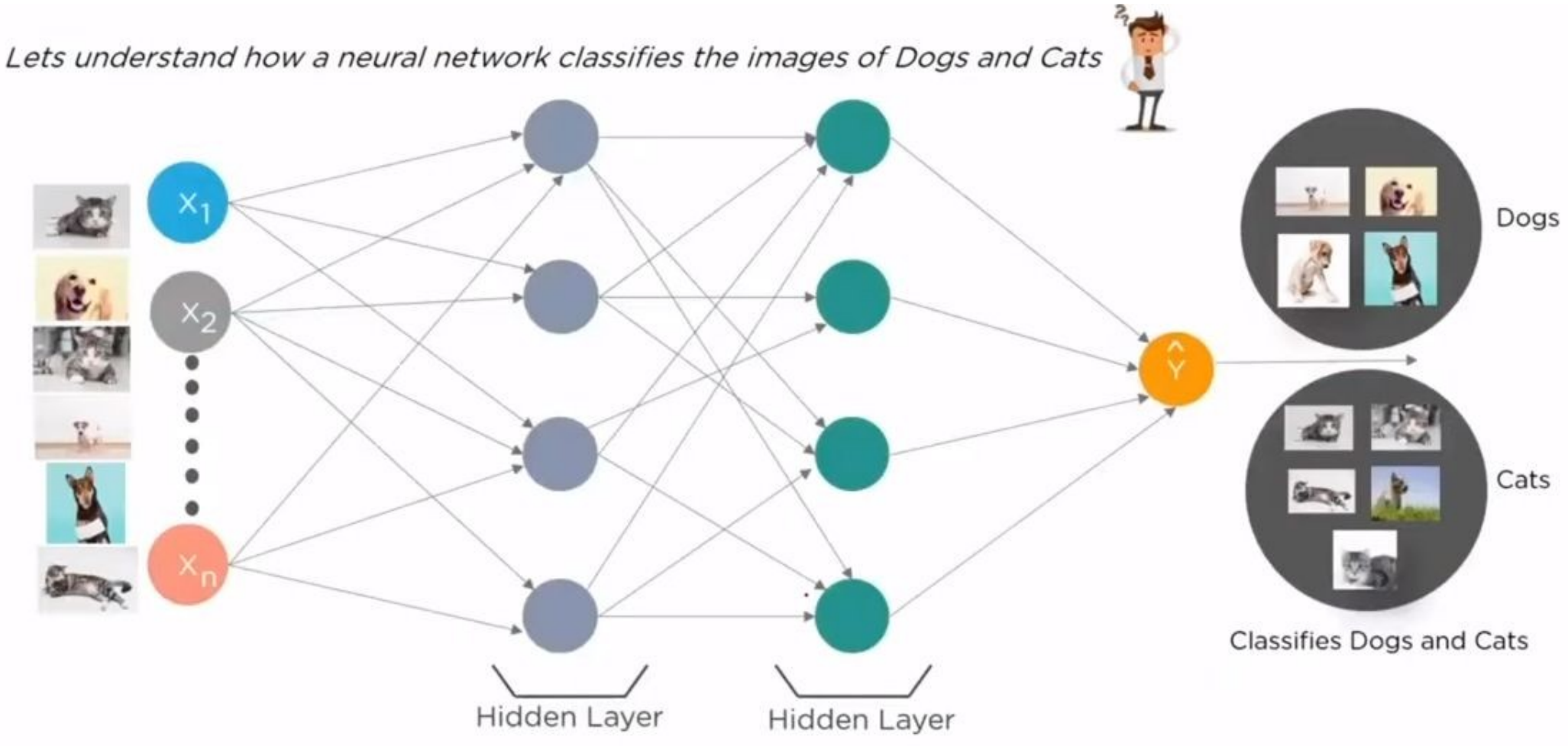


Working of a Neural Network

Lets understand how a neural network classifies the images of Dogs and Cats

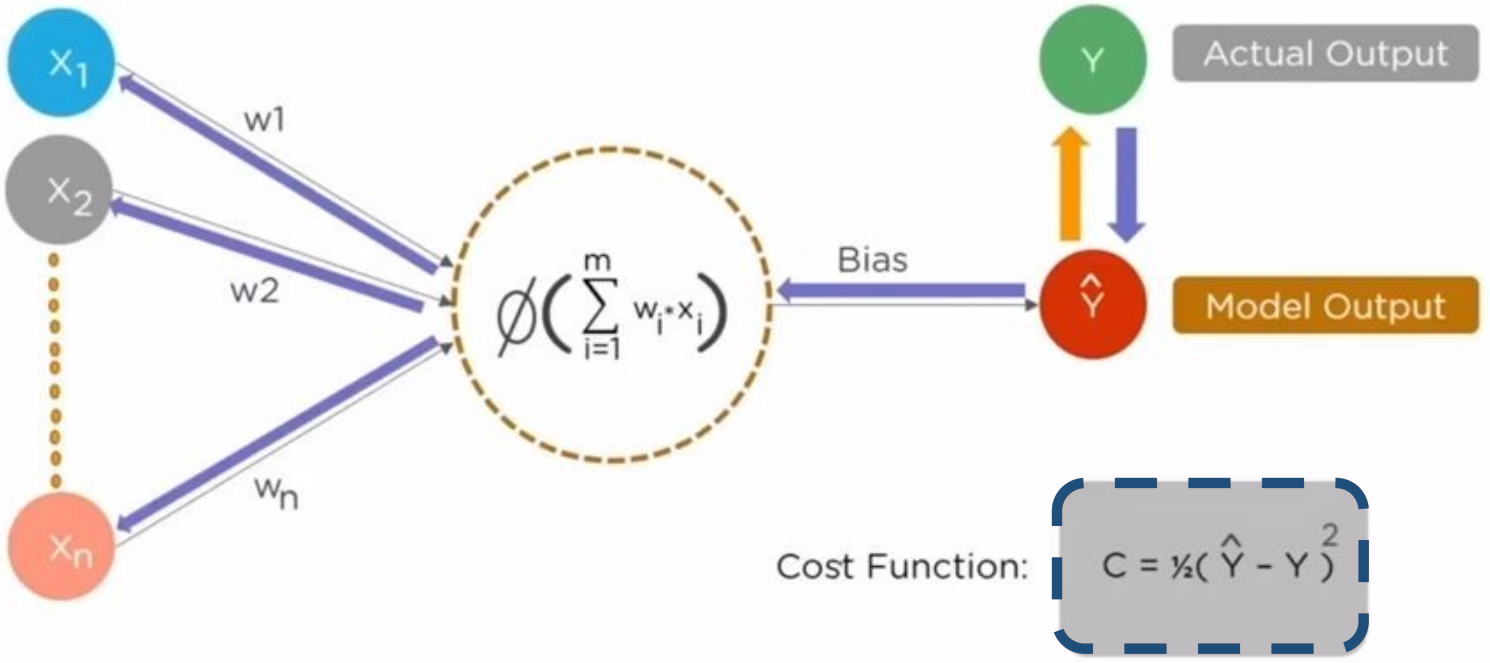


Working of a Neural Network

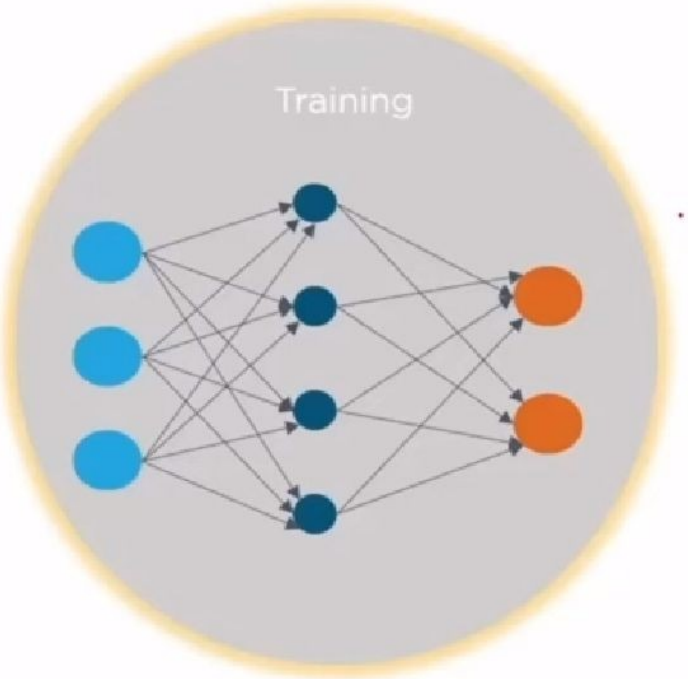


Working of a Neural Network

The *Cost* value is actually the difference between the Neural Nets predicted output and Actual output from a set of labeled training data. The least cost value is obtained by making adjustment to the weights and bias iteratively throughout the training process



Why are Deep Neural Networks Hard to Train

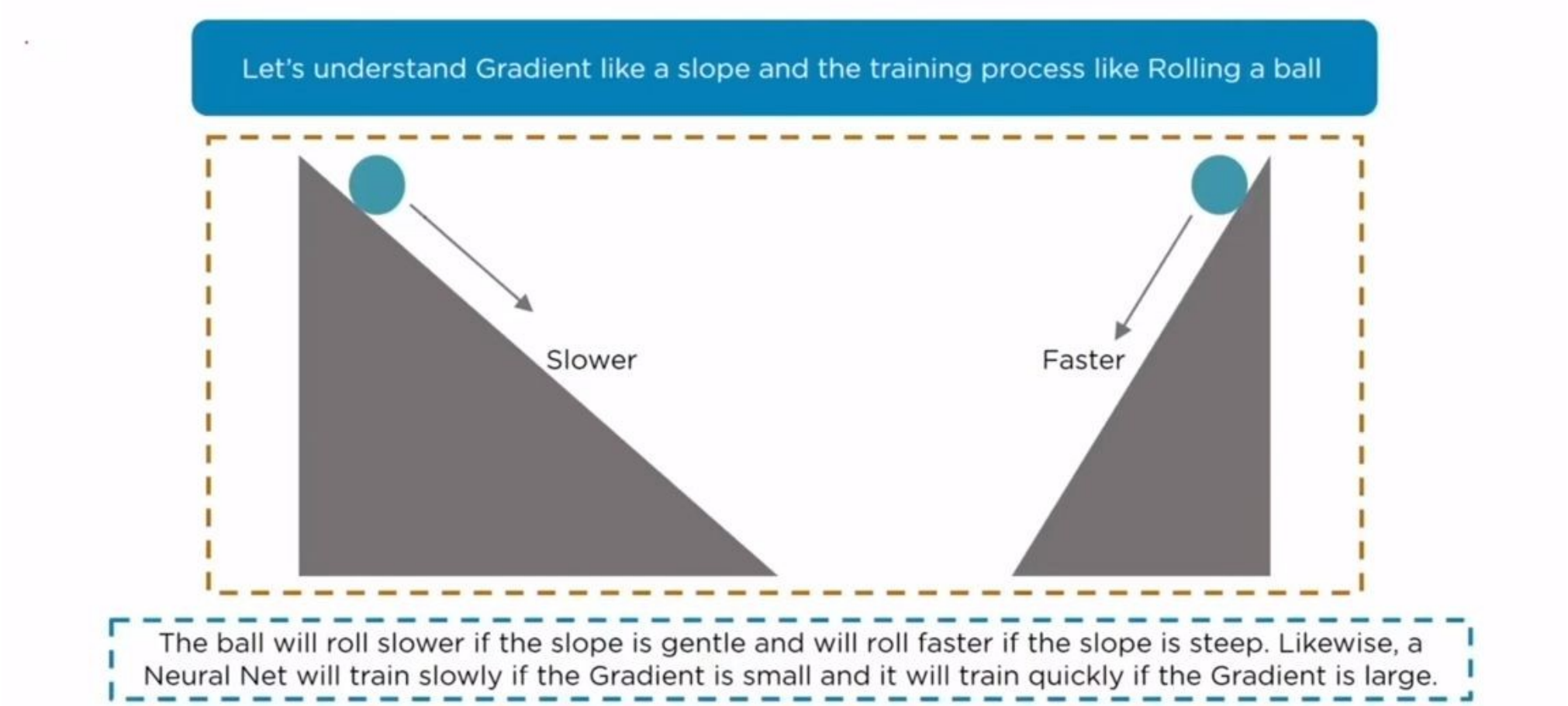


Gradient is the rate at which cost changes with respect to weight and bias

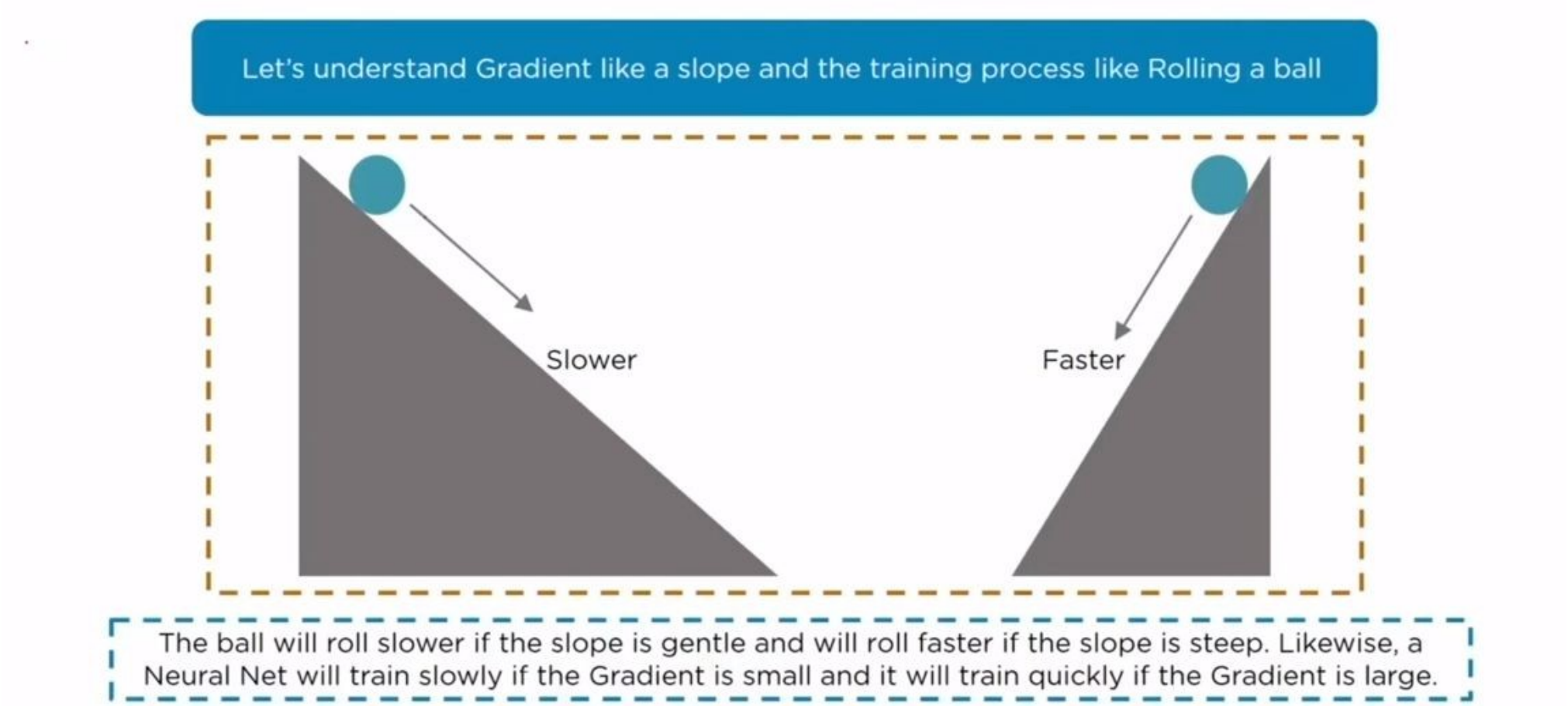
Until 2006, there was no proper method to accurately train deep neural networks due to a basic problem with the training process:

The Vanishing Gradient

Why are Deep Neural Networks Hard to Train

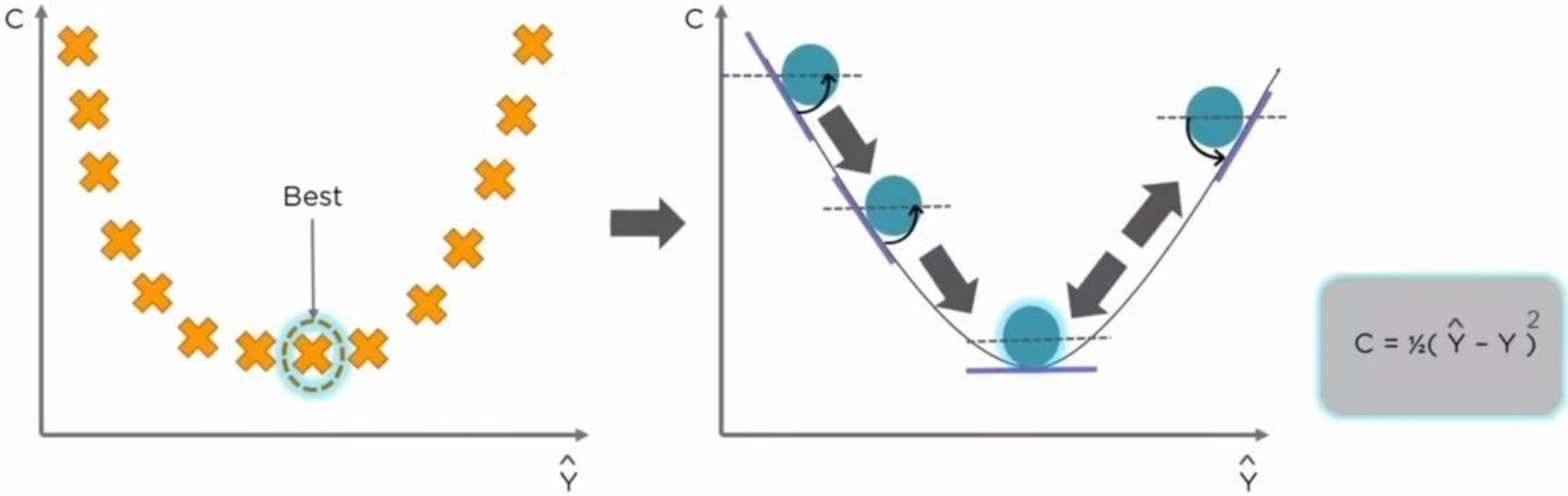


Why are Deep Neural Networks Hard to Train

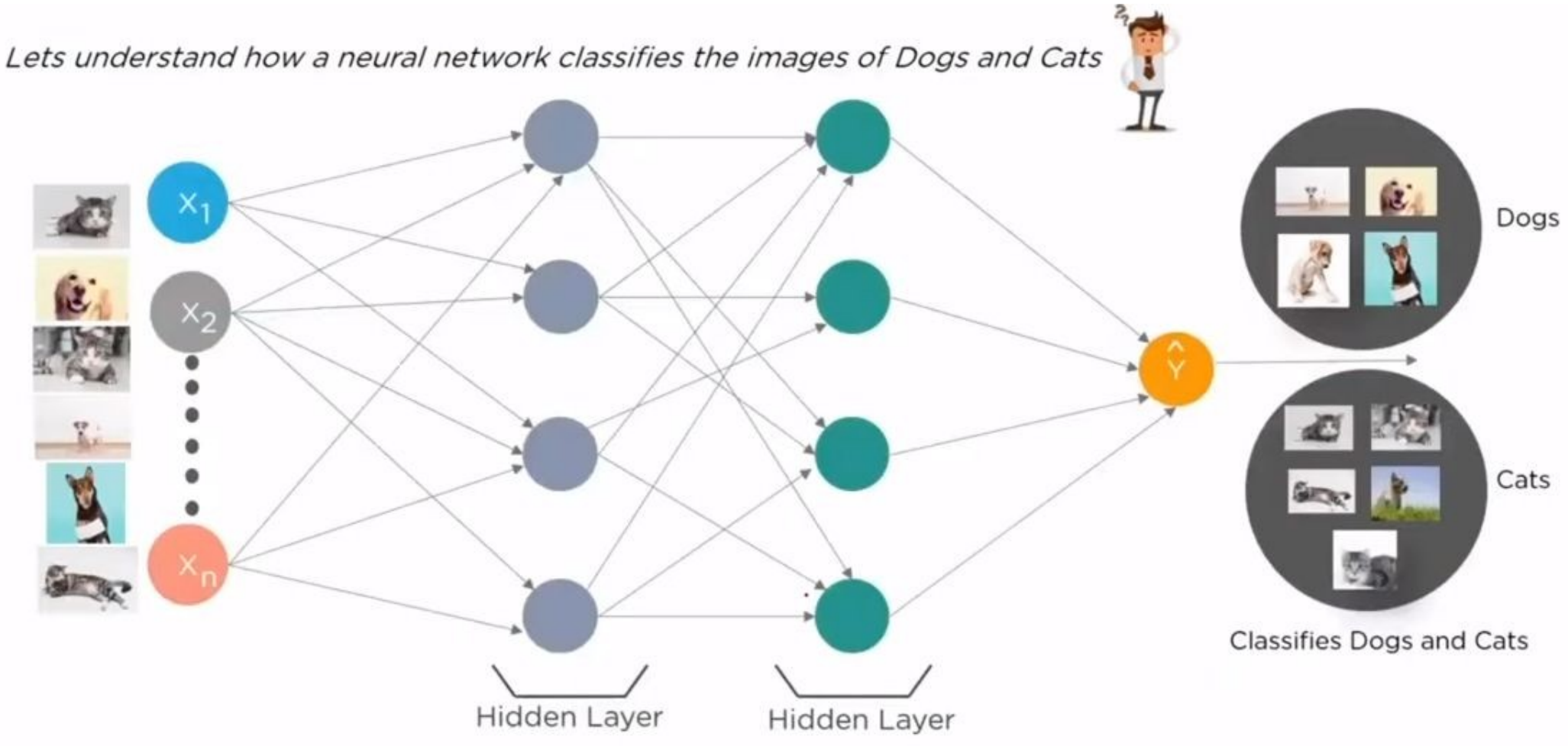


Why are Deep Neural Networks Hard to Train

Gradient Descent is an optimization algorithm for finding the minimum of a function



Working of a Neural Network



Gradient Descent Rule

The delta training rule is best understood by considering the task of training an *unthresholded* perceptron; that is, a *linear unit* for which the output *o* is given by

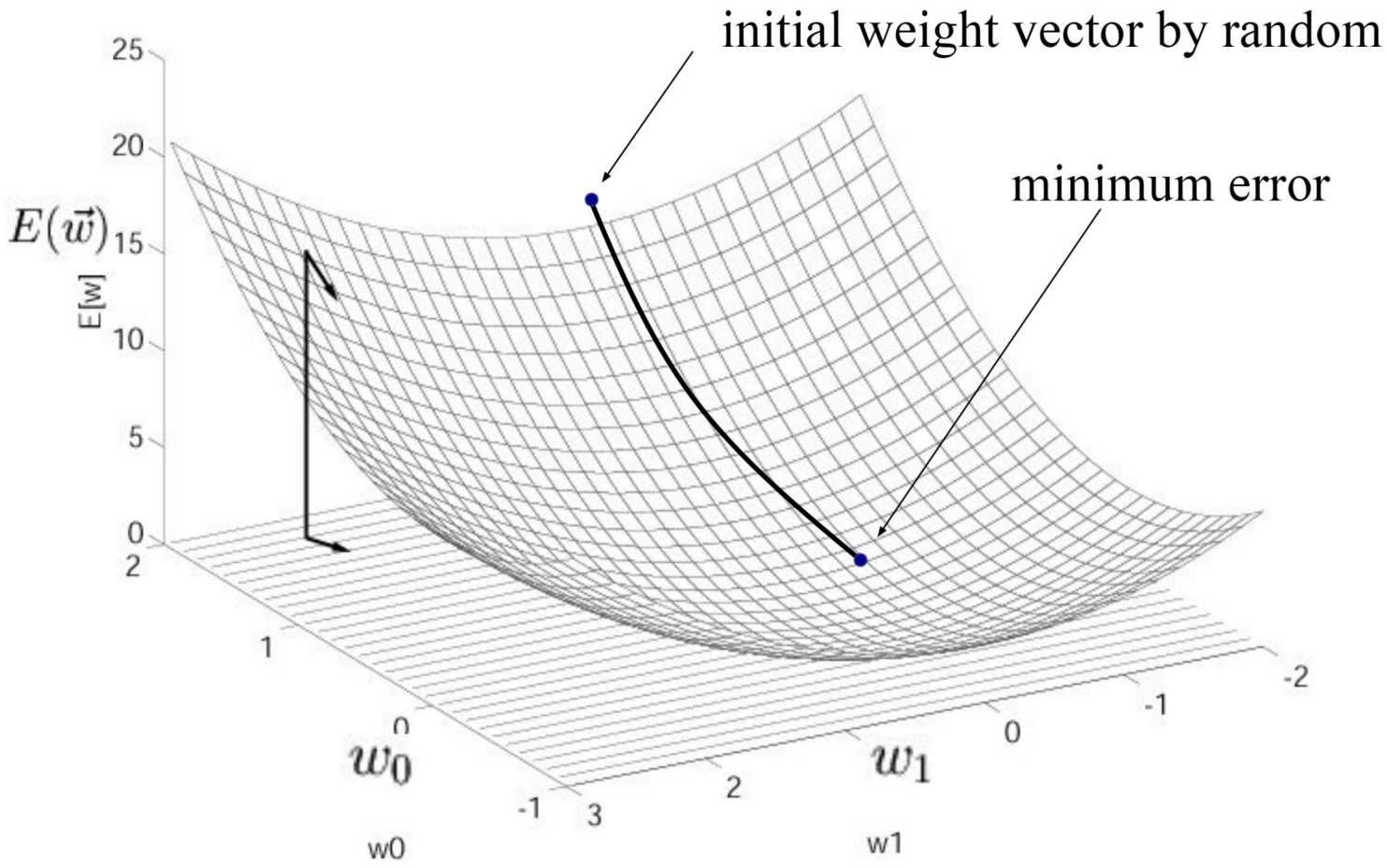
$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$

In order to derive a weight learning rule for linear units, let us begin by specifying a measure for the training error of a hypothesis (weight vector), relative to the training examples.

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

Where as ‘d’ is the number of outputs

Visualizing the Hypothesis Space



Derivation of the Gradient Descent Rule

The vector derivative is called the *gradient* of E with respect to $\vec{w}$, written $\nabla E(\vec{w})$

$$\nabla E(\vec{w}) \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots, \frac{\partial E}{\partial w_n}\right]$$

The gradient specifies the direction that produces the steepest increase in E. The negative of this vector therefore gives the direction of steepest decrease. The training rule for gradient descent is

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$$

where

$$\Delta\vec{w} = -\eta\nabla E(\vec{w})$$

Derivation of the Gradient Descent Rule (cont.)

The negative sign is presented because we want to move the weight vector in the direction that *decreases* E. This training rule can also be written in its component form

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

which makes it clear that steepest descent is achieved by altering each component w_i of $\vec{w}$ in proportion to $\frac{\partial E}{\partial w_i}$.

Derivation of the Gradient Descent Rule (cont.)

The vector of $\frac{\partial E}{\partial w_i}$ derivatives that form the gradient can be obtained by differentiating E

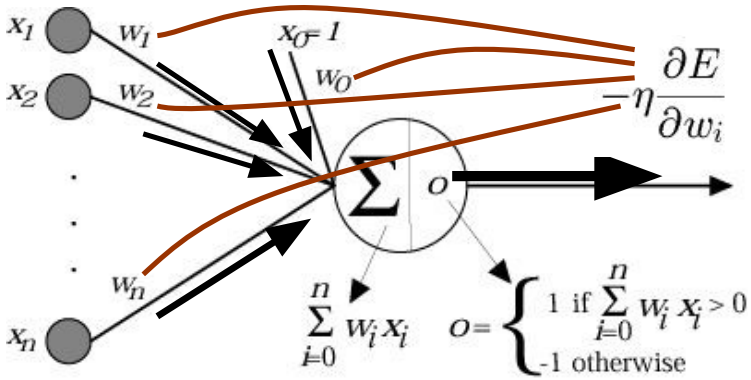
$$\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i}\frac{1}{2}\sum_{d\in D}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_{d\in D}\frac{\partial}{\partial w_i}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_{d\in D}2(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - o_d) \\
&= \sum_{d\in D}(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - \vec{w}\cdot\vec{x}_d) \\
\frac{\partial E}{\partial w_i} &= \sum_{d\in D}(t_d - o_d)(-x_{id})
\end{aligned}$$

The weight update rule *for standard gradient descent* can be summarized as

$$\Delta w_i = -\eta\frac{\partial E}{\partial w_i}$$

$$\Delta w_i = \eta\sum_{d\in D}(t_d - o_d)x_{id}$$

Stochastic Approximation to Gradient Descent



GRADIENT-DESCENT($training\_examples, \eta$)

Each training example is a pair of the form $\langle \vec{x}, t \rangle$, *where* $\vec{x}$ *is the vector of input values, and* t *is the target output value.* η *is the learning rate (e.g., .05).*

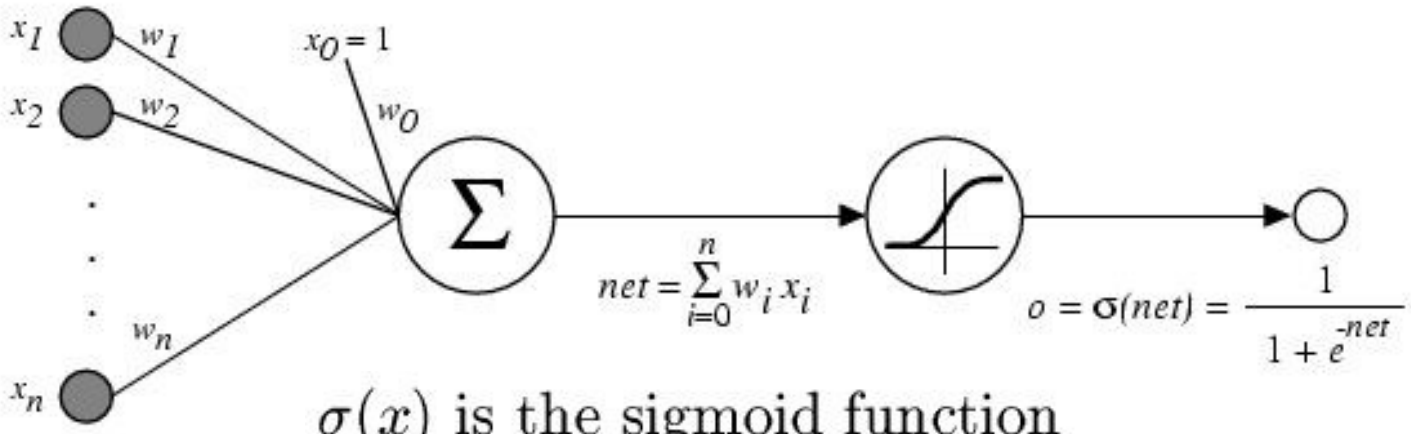
- Initialize each w_i to some small random value
- Until the termination condition is met, Do
  - Initialize each Δw_i to zero.
  - For each $\langle \vec{x}, t \rangle$ in *training_examples*, Do
    - Input the instance $\vec{x}$ to the unit and compute the output o
    - For each linear unit weight w_i, Do

$$\Delta w_i \leftarrow \Delta w_i + \eta(t - o)x_i$$

  - For each linear unit weight w_i, Do

$$w_i \leftarrow w_i + \Delta w_i$$

Backpropagation Algorithm



$\sigma(x)$ is the sigmoid function

$$\frac{1}{1 + e^{-x}}$$

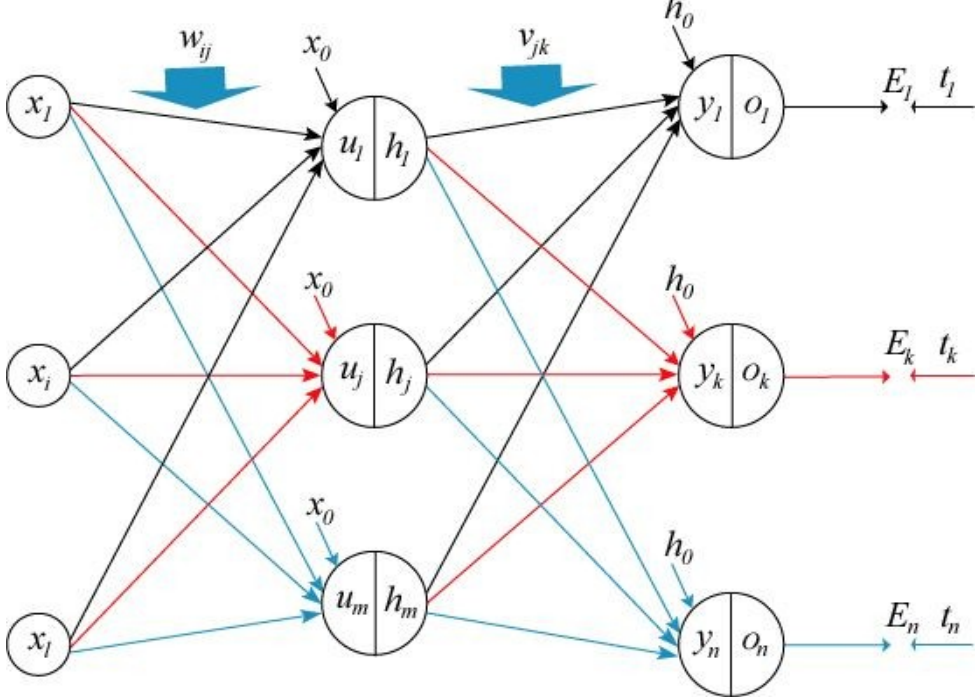
Error Function

The Backpropagation algorithm learns the weights for a multilayer network, given a network with a fixed set of units and interconnections. It employs gradient descent to attempt to minimize the squared error between the network output values and the target values for those outputs. We begin by redefining E to sum the errors over all of the network output units

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in outputs} (t_{kd} - o_{kd})^2$$

where *outputs* is the set of output units in the network, and t_{kd} and o_{kd} are the target and output values associated with the k^{th} output unit and training example d.

Architecture of Backpropagation



$$u_j = \sum_{i=0}^{l} w_{ij} \cdot x_i, \quad h_j = \frac{1}{1+e^{-u_j}}, 1 \le j \le m$$

$$y_k = \sum_{j=0}^{m} v_{jk} \cdot h_j, \quad o_k = \frac{1}{1+e^{-y_k}}, 1 \le k \le n$$

$$v_{jk}^{(new)} = v_{jk}^{(old)} + \eta \cdot \delta o_k \cdot h_j, 0 \le j \le m, 1 \le k \le n$$

$$w_{ij}^{(new)} = w_{ij}^{(old)} + \eta \cdot \delta h_j \cdot x_i, 0 \le i \le l, 1 \le j \le m$$

Backpropagation Learning Algorithm

Step 1. Initialize the weights of the perceptron randomly with number between -0.1 and 0.1.

$$w_{ij} = random([-0.1, 0.1]) \; 0 \leq i \leq l, 1 \leq j \leq m$$

$$v_{jk} = random([-0.1, 0.1]) \; 0 \leq j \leq m, 1 \leq k \leq n$$

Step 2. Present the pattern $p = (x_1, x_2, x_i, \cdots, x_l) \in \mathbb{R}^l$ to the perceptron, $p \in P$

Backpropagation Learning Algorithm

Step 3. Compute the values of the hidden-layer nodes using the formula

$$u_j = \sum_{i=0}^{l} w_{ij} \cdot x_i, \quad h_j = \frac{1}{1 + e^{-u_j}}, 1 \leq j \leq m$$

Step 4. Calculate the values of the output nodes using the formula

$$y_k = \sum_{j=0}^{m} v_{jk} \cdot h_j, \quad o_k = \frac{1}{1 + e^{-y_k}}, 1 \leq k \leq n$$

Backpropagation Learning Algorithm

Step 5. Compute the errors $\delta o_k, 1 \leq k \leq n$, in the output layer using

$$\delta o_k = (t_k - o_k) o_k (1 - o_k), 1 \leq k \leq n$$

Step 6. Compute the errors $\delta h_j, 1 \leq j \leq m$, in the hidden layer using

$$\delta h_j = h_j (1 - h_j) \sum_{k=1}^{n} \delta o_k \cdot v_{jk}, 1 \leq j \leq m$$

Backpropagation Learning Algorithm

Step 7. Adjust the weights between the output layer and the hidden layer according to the formula

$$v_{jk}^{(new)} = v_{jk}^{(old)} + \eta \cdot \delta o_k \cdot h_j, 0 \leq j \leq m, 1 \leq k \leq n$$

Step 8. Adjust the weights between the hidden layer and the input layer according to

$$w_{ij}^{(new)} = w_{ij}^{(old)} + \eta \cdot \delta h_j \cdot x_i, 0 \leq i \leq l, 1 \leq j \leq m$$

Backpropagation Learning Algorithm

- Repeat Step 2 through 8 for each element of training set.
- Unitll a maximum number of iteration or the error reduce to zero.

Example: The XOR problem:

- Single hidden layer: 3 Sigmoid neurons
- 2 inputs, 1 output

	x1	x2	y
Example 1	0	0	**0**
Example 2	0	1	**1**
Example 3	1	0	**1**
Example 4	1	1	**0**

Desired I/O table (XOR):

Example: The XOR problem:

initial_weights =

0.0654 0.2017 0.0769 0.1782 0.0243
0.0806 0.0174 0.1270 0.0599 0.1184
0.1335 0.0737 0.1511

final_weights =

4.6970 -4.6585 2.0932 5.5168 -5.7073
-3.2338 -0.1886 1.6164 -0.1929 -6.8066
6.8477 -1.6886 4.1531

Example: The XOR problem:

Mapping produced by the trained neural net:

	x1	x2	y
Example 1	0	0	0.0824
Example 2	0	1	0.9095
Example 3	1	0	0.9470
Example 4	1	1	0.0464

Remember.....

- ANNs perform well, generally better with larger number of hidden units
- More hidden units generally produce lower error
- Determining network topology is difficult
- Choosing single learning rate impossible
- Difficult to reduce training time by altering the network topology or learning parameters
- NN(Subset) often produce better results